@test_long_term_learning

Appears in: **DL-07**

tests/test_long_term_learning.py:36-46, 174-336

"Build a standalone integration test — @test_long_term_learning — that runs two complete pipeline calls with a structured set of deliveries between them." … "Run @test_long_term_learning to produce side-by-side evidence of delivery history changing the recommendation …"

@test_long_term_learning

tests/test_long_term_learning.py:36-46

```python
ROUNDS = [
    # (supplier_id, sku_id, lateness_days, label)
    ("SUP-003", "SKU-003", 8, "QuickShip  8d late  #1"),
    ("SUP-003", "SKU-003", 8, "QuickShip  8d late  #2"),
    ("SUP-003", "SKU-003", 8, "QuickShip  8d late  #3"),
    ("SUP-003", "SKU-003", 8, "QuickShip  8d late  #4"),
    ("SUP-003", "SKU-003", 8, "QuickShip  8d late  #5"),
    ("SUP-001", "SKU-003", 0, "FastParts  on-time  #1"),
    ("SUP-001", "SKU-003", 0, "FastParts  on-time  #2"),
    ("SUP-001", "SKU-003", 0, "FastParts  on-time  #3"),
]
```

…

tests/test_long_term_learning.py:174-336

```python
def run():
    W = 70
    print()
    print("=" * W)
    print("Betsy Long-Term Learning Test")
    print(f"Server: {BASE}")
    print("=" * W)
    print("Two real LLM runs with 8 EMA delivery rounds between them.")
    print("QuickShip gets 5 x 8d-late; FastParts gets 3 x on-time.")
    print("Predicted: FastParts overtakes QuickShip for SKU-003.")
    print(f"Runtime: ~3-5 min\n")

    # ── Phase 0: Preflight ──────────────────────────────────────
    print("[0] Preflight")
    try:
        assert httpx.get(f"{BASE}/health", timeout=5).json().get("status") == "ok"
    except Exception:
        print(f"    ERROR: server not reachable at {BASE}")
        return

    post("/api/scenario/reset")
    time.sleep(1)

    sups = supplier_scores()
    baseline = {sid: sups[sid]["reliability_score"] for sid in FOCUS}
    print(f"    Server OK.  Baseline: " +
          ", ".join(f"{sups[sid]['name']}={v:.4f}" for sid, v in baseline.items()))
    print(f"    Composite:  " +
          ", ".join(f"{sups[sid]['name']}={composite(v, LEAD_DAYS[sid]):.4f}"
                    for sid, v in baseline.items()))

    # ── Phase 1: Run #1 — baseline scores ──────────────────────
    print()
    print("[1] Pipeline Run #1 — stockout_warning, baseline scores")
    post("/api/scenario/stockout_warning")
    time.sleep(0.5)

    try:
        approvals_1, pl1 = run_pipeline("run1")
    except TimeoutError as e:
        print(f"    {e}")
        return

    print_pipeline_summary(pl1)

    appr_1 = chosen_supplier(approvals_1)
    sid_1 = appr_1["supplier_id"] if appr_1 else None
    name_1 = sups.get(sid_1, {}).get("name", sid_1) if sid_1 else "none queued"
    print(f"   Chosen supplier: {name_1} ({sid_1})")
    if not appr_1:
        print(f"   NOTE: no approval queued — pipeline may have auto-approved or found no action.")
        print(f"         New approvals this run: {len(approvals_1)}")

    # ── Phase 2: Delivery simulation ────────────────────────────
    print()
    print("[2] Delivery Simulation — 8 EMA rounds")
    print(f"   {'#':<4} {'Label':<28} {'Before':>8}  {'After':>8}  {'Perf':>6}")
    print("   " + "-" * 58)

    for i, (supplier_id, sku_id, lateness, label) in enumerate(ROUNDS, 1):
        perf = max(0.0, 1.0 - lateness * 0.1)
        score_before = supplier_scores()[supplier_id]["reliability_score"]
        simulate_delivery(supplier_id, sku_id, lateness)
        score_after = supplier_scores()[supplier_id]["reliability_score"]
        print(f"   {i:<4} {label:<28} {score_before:>8.4f}  {score_after:>8.4f}  {perf:>6.1f}")

    sups_after = supplier_scores()
    learned = {sid: sups_after[sid]["reliability_score"] for sid in FOCUS}

    print()
    print("   Score summary (focus suppliers):")
    print(f"   {'Supplier':<22} {'Baseline':>9}  {'Learned':>9}  {'Delta':>7}  {'Composite'}")
    print("   " + "-" * 63)
    for sid in FOCUS:
        name = sups[sid]["name"]
        b = baseline[sid]
        a = learned[sid]
        lead = LEAD_DAYS[sid]
        delta = a - b
        sign = "+" if delta >= 0 else ""
        print(f"   {name:<22} {b:>9.4f}  {a:>9.4f}  {sign}{delta:>6.4f}  "
              f"{composite(b, lead):.4f} -> {composite(a, lead):.4f}")

    # ── Phase 3: Run #2 — learned scores explicitly restored ────
    print()
    print("[3] Pipeline Run #2 — same scenario, learned scores restored")

    # Inject scenario (this resets supplier scores to baseline)
    post("/api/scenario/stockout_warning")
    time.sleep(0.5)

    # Explicitly restore learned scores via PATCH endpoint
    restore_scores(learned)

    # Verify
    live = supplier_scores()
    for sid in FOCUS:
        expected = learned[sid]
        actual = live[sid]["reliability_score"]
        ok = abs(actual - expected) < 0.0001
        print(f"   Score restored {sups[sid]['name']}: {actual:.4f}  ({'OK' if ok else 'MISMATCH'})")

    try:
        approvals_2, pl2 = run_pipeline("run2")
    except TimeoutError as e:
        print(f"    {e}")
        return

    print_pipeline_summary(pl2)

    appr_2 = chosen_supplier(approvals_2)
    sid_2 = appr_2["supplier_id"] if appr_2 else None
    name_2 = sups.get(sid_2, {}).get("name", sid_2) if sid_2 else "none queued"
    print(f"   Chosen supplier: {name_2} ({sid_2})")
    if not appr_2:
        print(f"   NOTE: no approval queued — new approvals: {len(approvals_2)}")

    # ── Phase 4: Summary ────────────────────────────────────────
    print()
    print("=" * W)
    print("[4] Summary")
    print("=" * W)

    print(f"\nComposite score shift for SKU-003 (score / lead_days):\n")
    print(f"   {'Supplier':<22} {'Before comp':>12}  {'After comp':>11}  {'Direction'}")
    print("   " + "-" * 57)
    for sid in sorted(FOCUS, key=lambda s: composite(baseline[s], LEAD_DAYS[s]), reverse=True):
        name = sups[sid]["name"]
        lead = LEAD_DAYS[sid]
        cb = composite(baseline[sid], lead)
        ca = composite(learned[sid], lead)
        direction = "up" if ca > cb else "DOWN"
        print(f"   {name:<22}  {cb:.4f}/{lead}d      {ca:.4f}/{lead}d   {direction}")

    print(f"\nPipeline decision:")
    print(f"   Run #1 (baseline):        {name_1}  ({sid_1})")
    print(f"   Run #2 (learned scores):  {name_2}  ({sid_2})")

    if sid_1 and sid_2:
        if sid_1 != sid_2:
            print(f"\n   SUPPLIER SWITCH CONFIRMED")
            print(f"   Betsy switched from {name_1} to {name_2}.")
            print(f"   Score learning directly changed the procurement decision.")
        else:
            comp_sid_1 = composite(learned.get("SUP-001", baseline["SUP-001"]), LEAD_DAYS["SUP-001"])
            comp_sid_3 = composite(learned.get("SUP-003", baseline["SUP-003"]), LEAD_DAYS["SUP-003"])
            print(f"\n   Same supplier both runs.")
            print(f"   Final composites: FastParts={comp_sid_1:.4f}, QuickShip={comp_sid_3:.4f}")
            if comp_sid_3 > comp_sid_1:
                print(f"   QuickShip composite still higher — retained correctly.")
            else:
                print(f"   FastParts composite higher — check LLM reasoning in /api/agent-log.")
    else:
        if not sid_1 or not sid_2:
            print(f"\n   Could not compare: one or both runs had no supplier in approval.")
            print(f"   Check /api/agent-log for pipeline decisions.")

    print(f"\nEMA formula verified across 8 delivery rounds:")
    print(f"   new = 0.2 * performance + 0.8 * old")
    print(f"   performance = max(0, 1.0 - lateness_days * 0.1)")
    print(f"   QuickShip: {baseline['SUP-003']:.4f} -> {learned['SUP-003']:.4f}  (5 x 8d-late)")
    print(f"   FastParts: {baseline['SUP-001']:.4f} -> {learned['SUP-001']:.4f}  (3 x on-time)")
    print("=" * W + "\n")
```

What it shows: The integration test: run #1 (baseline) → 8 deliveries (QuickShip 5×8-days-late, FastParts 3×on-time) → restore learned scores → run #2, then compare the chosen supplier.

Source: tests/test_long_term_learning.py:36-46, 174-336 · image rendered by build_snippets.py